

Relatório Técnico - Aura Rank

Sistema Gerenciador de Usuários com Pontuação de Aura - Programação Web 2

1. Objetivo do Sistema

O Aura Rank é um sistema web CRUD desenvolvido em PHP vanilla com banco MySQL containerizado via Docker. Aplicando conceitos de gamificação com temática Dragon Ball Z, gerencia usuários cuja pontuação determina automaticamente sua categoria de "aura" (Negativa, NPC, Pura, +Ego, Lendária). O sistema oferece cadastro, listagem com filtros, edição e exclusão tanto de usuários quanto de categorias, além de histórico de alterações de pontuação e sistema de conquistas. A integridade dos dados é garantida por Foreign Keys em todas as tabelas relacionadas.

2. Modelagem do Banco de Dados

O schema db_users possui 5 tabelas normalizadas. O script banco.sql contém a DDL completa e dados iniciais (seed) que são carregados automaticamente pelo Docker na primeira execução.

2.1 Estrutura das Tabelas

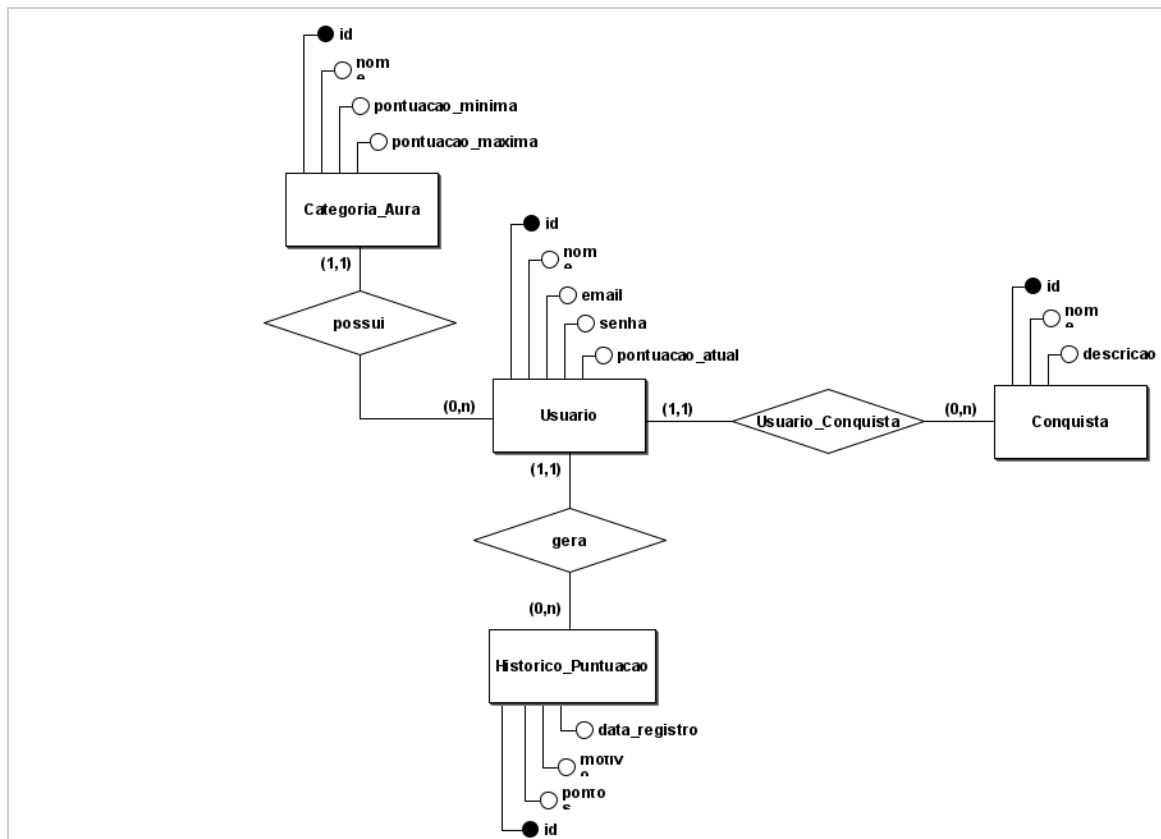
Tabela	Colunas	Descrição
categoria_aura	id (PK), nome, pontuacao_minima, pontuacao_maxima	Define as 5 faixas de aura com intervalos de pontuação
usuario	id (PK), nome, email, senha, pontuacao_atual, fk_categoria_aura_id (FK)	Tabela principal. FK referencia a categoria correspondente à pontuação
historico_pontuacao	id (PK), motivo, data_registro, pontos, fk_usuario_id (FK)	Log de todas as alterações de pontuação com motivo e timestamp
conquista	id (PK), nome, descricao	Catálogo de conquistas desbloqueáveis
usuario_conquista	fk_usuario_id (PK,FK), fk_conquista_id (PK,FK)	Tabela associativa N:N com chave primária composta

2.2 Relacionamentos e Integridade Referencial

- categoria_aura (1:N) usuario:** FK fk_categoria_aura_id. Cada usuário pertence a exatamente uma categoria. Ao cadastrar ou editar um usuário, a categoria é atribuída automaticamente via buscarCategoriaIdPorPontuacao() usando BETWEEN na faixa de pontuação.
- usuario (1:N) historico_pontuacao:** FK fk_usuario_id. Registra cada mudança de pontuação com motivo e data.
- usuario (N:N) conquista:** Via tabela associativa usuario_conquista com PK composta (fk_usuario_id, fk_conquista_id), impedindo duplicatas.

As Foreign Keys garantem que o SGBD bloqueie operações inválidas: não é possível deletar uma categoria com usuários vinculados nem inserir um usuário referenciando categoria inexistente.

3. Diagrama Entidade-Relacionamento (DER)



4. Operações CRUD

O arquivo `processa.php` funciona como controller central da aplicação, recebendo todas as submissões de formulários (POST) e requisições de exclusão (GET). A tabela abaixo resume o fluxo completo:

Operação	Entidade Usuário	Entidade Categoria	HTTP
Create	<code>cadastro.php</code> envia nome, email, senha e pontuação para <code>processa.php</code>	<code>cadastro_categoria.php</code> envia nome, pontuação mínima e máxima	POST
Read	<code>index.php</code> lista todos os usuários com filtros combináveis (nome LIKE, categoria, faixa de pontuação)	<code>categorias.php</code> lista todas as categorias ordenadas por pontuação mínima	GET
Update	<code>editar.php?id=X</code> carrega dados do usuário e envia alterações para <code>processa.php</code>	<code>editar_categoria.php?id=X</code> carrega e envia alterações da categoria	POST
Delete	<code>processa.php?deletar=1&id=X</code> com confirmação <code>onclick="confirm()"</code>	<code>processa.php?deletar_categoria=1&id=X</code>	GET

O cálculo da categoria do usuário é automático: a função `buscarCategoriaIdPorPontuacao()` consulta `categoria_aura` com `WHERE :pontuacao BETWEEN pontuacao_minima AND pontuacao_maxima` e retorna o ID correspondente. A view `index.php` exibe o nome da categoria usando `calcularStatusAura()`.

5. Uso dos Métodos HTTP (GET e POST)

A aplicação diferencia corretamente os métodos HTTP conforme a semântica de cada operação:

- **GET:** usado para operações de leitura e exclusão. As buscas com filtros são enviadas via query string (`?nome=Marcos&categoria=2&pontuacao_min=100&pontuacao_max=1000`), permitindo que os resultados sejam compartilhados ou marcados como favoritos. A exclusão de registros também usa GET com parâmetro na URL e confirmação JavaScript para evitar acidentes. O feedback das operações é passado via `?salvar=OK` ou `?salvar=ERROR`.
- **POST:** utilizado exclusivamente para operações que modificam dados (cadastro e edição). Os dados do formulário, incluindo senhas, trafegam no corpo da requisição e não aparecem na URL nem no histórico do navegador. Campos ocultos (`<input type="hidden" name="inserir" value="1">`) sinalizam ao controller `processa.php` qual ação específica deve ser executada.

6. Execução do Sistema

```
# Iniciar o container MySQL (cria banco, tabelas e dados seed)
docker compose up -d

# Iniciar o servidor de desenvolvimento PHP
php -S localhost:8000

# Acessar: http://localhost:8000
```

Pré-requisitos: Docker Engine, PHP 8+ com extensão PDO MySQL habilitada.

7. Deploy e Tecnologias

O sistema é compatível com qualquer servidor que ofereça suporte a PHP e MySQL. Para publicar, importe banco.sql no banco de destino e configure os parâmetros de conexão em conexao.php (host, usuário, senha e nome do banco).

Camada	Tecnologia
Backend	PHP 8.5 (vanilla, sem frameworks ou dependências externas)
Banco de dados	MySQL 8.4 (Docker Compose, volume persistente)
Acesso a dados	PDO com Prepared Statements, charset utf8mb4
Frontend	HTML5 semântico + CSS3 inline
Modelagem	BRModelo (DER Conceitual e Modelo Lógico)

Fim do Relatório Técnico